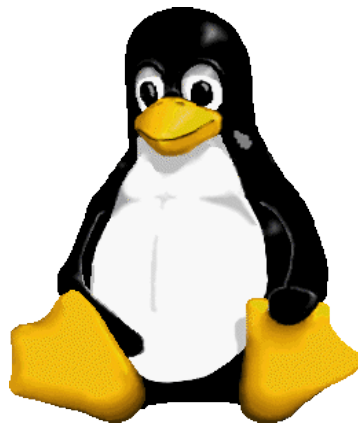


Protocolo de ligação de Dados



Redes de Computadores

Turma 14 Grupo 7

Trabalho realizado por:

Diogo Ramos(up202207954)

Tiago Pires(up202208910)

Sumário

O projeto realizado implementa o protocolo de ligação de dados para a transmissão de um ficheiro escolhido, utilizando a porta série RS-232.

Com a realização deste projeto, ficamos a perceber melhor como funciona a transmissão de ficheiros através de uma porta série e também algoritmos como o STOP&WAIT, byte stuffing e destuffing, que abordámos nas aulas.

Introdução

O objetivo do trabalho foi desenvolver o protocolo de ligação de dados para transferir um ficheiro, seguindo o guião que nos foi disponibilizado. O relatório da elaboração deste trabalho está dividido pelas seguintes partes:

- **Arquitetura:** Blocos funcionais e Interfaces;
- **Estrutura do código:** Principais Api's , estruturas de dados e funções;
- **Casos de uso principais:** Identificação e sequência de chamadas de funções;
- **Lógica do protocolo de ligação:** Principais aspetos funcionais e estratégia de implementação;
- **Protocolo de Aplicação:** Principais aspetos funcionais e estratégia de implementação;
- **Validação:** Testes realizados;
- **Eficiência do protocolo de ligação de dados:** Eficiência da implementação do algoritmo Stop&Wait abordado na aula teórica;
- **Conclusões:** Síntese e reflexão sobre os resultados obtidos durante todo o projeto;

Arquitetura

Blocos Funcionais

O nosso projeto está dividido em 2 pilares fundamentais, **ApplicationLayer** e **LinkLayer**, com um auxiliar essencial a **SerialPort** que permite fazer toda a ligação que é preciso para a porta série funcionar corretamente.

A implementação da **ApplicationLayer** encontra-se nos ficheiros **application_layer.h** e **application_layer.c** , utiliza as funções implementadas na **Linklayer** para a transferência e receção de pacotes dos dados de um ficheiro, e também outras funções adicionais, por exemplo para criar pacotes de dados e pacotes de controlo.

A implementação da **Linklayer** encontra-se nos ficheiros **link_layer.h** e **link_layer.c** , e a camada de ligação de dados serve não só para iniciar, como também para terminar a ligação, e é ainda fulcral para criar, enviar e validar tramas de dados através da porta série.

Interfaces

Emissor

```
Starting link-layer protocol application
- Serial port: /dev/ttyS10
- Role: tx
- Baudrate: 9600
- Number of tries: 3
- Timeout: 4
- Filename: penguin.gif
Initializing applicationLayer with role: tx, baudRate: 9600, filename: penguin.gif
Role first character: t
Role set to LLTx (Transmitter)
```

Recetor

```
Starting link-layer protocol application
- Serial port: /dev/ttyS11
- Role: rx
- Baudrate: 9600
- Number of tries: 3
- Timeout: 4
- Filename: penguin-received.gif
Initializing applicationLayer with role: rx, baudRate: 9600, filename: penguin-received.gif
Role first character: r
Role set to LLRx (Receiver)
```

Estrutura do código

ApplicationLayer

Na *Applicationlayer* não foi preciso estruturas auxiliares de dados, usámos apenas as funções da imagem e as da *LinkLayer*.

```
// Application layer main function.
// Arguments:
//   serialPort: Serial port name (e.g., /dev/ttyS0).
//   role: Application role {"tx", "rx"}.
//   baudrate: Baudrate of the serial port.
//   nTries: Maximum number of frame retries.
//   timeout: Frame timeout.
//   filename: Name of the file to send / receive.

void applicationLayer(const char *serialPort, const char *role, int baudRate,
                    int nTries, int timeout, const char *filename);

// Send a data packet.
int sendDtPckt(unsigned char* data, int dataSize);

// Read a control packet.
int readctrlpckt(unsigned char control, unsigned char* buf, size_t* file_size, char* filename);

// Send a control packet.
int sendCtrlPckt(int some, const char* fileName, long int length);
```

LinkLayer

Na camada de ligação, foram usadas duas estruturas de dados auxiliares, **LinkLayerRole** que serve para interpretar se a role é transmissor ou recetor, **LinkLayer** onde estão definidos atributos necessários para a transferência dos dados ser realizada corretamente. As funções implementadas foram as necessárias apenas, onde **readControlFrame()** serve para perceber se a trama foi bem recebida ou não (RR ou REJ).

```
typedef enum
{
    LLTx,
    LLRx,
} LinkLayerRole;

typedef struct
{
    char serialPort[50];
    LinkLayerRole role;
    int baudRate;
    int nRetransmissions;
    int timeout;
} LinkLayer;

// SIZE of maximum acceptable payload.
// Maximum number of bytes that application layer should send to link layer
#define MAX_PAYLOAD_SIZE 1000

// MISC
#define FALSE 0
#define TRUE 1

// Open a connection using the "port" parameters defined in struct linkLayer.
// Return "1" on success or "-1" on error.
int llopen(LinkLayer connectionParameters);

// Send data in buf with size bufSize.
// Return number of chars written, or "-1" on error.
int llwrite(const unsigned char *buf, int bufSize);

// Receive data in packet.
// Return number of chars read, or "-1" on error.
int llread(unsigned char *packet);

// Close previously opened connection.
// if showStatistics == TRUE, link layer should print statistics in the console on close.
// Return "1" on success or "-1" on error.
int llclose(int showStatistics);

// Handler for alarm signal.
void alarmHandler(int signal);

// Read a control frame from the serial port.
unsigned char readControlFrame ();
```

Casos de uso Principais

Recetor

- 1) **llopen()**- Responsável pelo início da conexão com o transmissor, através da troca de UA e SET frames, e pela conexão com a porta série.
- 2) **readctrlpkt()**- Função responsável por ler o pacote de controlo inicial, para saber quando é que o programa tem de começar a enviar pacotes de dados.
- 3) **llread()**- Basicamente é uma máquina de estados que analisa e valida as tramas enviadas pelo llwrite() e seguidamente envia uma trama para ser analisada pelo llwrite() (RR ou REJ).
- 4) **llclose()**- Responsável pelo fim da conexão com o transmissor, através da troca de DISC frames, e pelo fim da conexão com a porta série.

Emissor

- 1) **llopen()**-Responsável pelo início da conexão com o recetor, através da troca de UA e SET frames, e pela conexão com a porta série.
- 2) **sendctrlpckt()**- Responsável por enviar pacotes de controlo para o recetor posteriormente perceber quando é o início e o fim dos pacotes de dados.
- 3) **senddtpckt()**- Responsável pelo envio de pacotes de dados do ficheiro pretendido, através da chamada à função **llwrite()**.
- 4) **llwrite()**- Envia tramas de informação com base no pacote que é recebido através da porta série para o recetor.
- 5) **readcontrolframe()**- Máquina de estados que recebe e analisa a resposta do recetor.
- 6) **llclose()**- Responsável pelo fim da conexão com o recetor, através da troca de DISC frames, e pelo fim da conexão com a porta série.

Lógica do protocolo de ligação

A camada de ligação serve para comunicar diretamente com a porta série e assim garantir a comunicação entre o emissor e o recetor.

O passo de iniciar a conexão com a porta série, é realizado pelo **llopen()**. Seguidamente a abrir a conexão da porta série com **openserialport()**, o emissor envia um frame SET e espera a resposta do recetor que se tudo estiver corretamente implementado será um frame UA, já o recetor faz o contrário espera o frame SET e envia o UA. Se toda esta conexão for bem sucedida o emissor começa a enviar tramas de informação.

As tramas de informação são enviadas pelo emissor através do uso do **llwrite()**. A função recebe como argumento um pacote e o tamanho do pacote e com esta informação, vai criar uma trama de informação depois de aplicar byte stuffing ao pacote que recebe, de forma a evitar error com bytes que sejam iguais à FLAG ou ao ESC da trama. Depois de enviar esta trama de informação para o recetor, o emissor espera a resposta, e se esta for rejeitada, vai tentar enviar novamente, ou até ser aceite ou até exceder o número de retransmissões, se for aceite passa para a próxima trama de informação, e é assim até não ter mais dados para enviar.

A leitura por sua vez é realizada pelo **llread()**, que basicamente serve para ler a trama de informação enviada, realizando um destuff dos dados, e se não houver nada de errado, retorna receiver ready, o que permite ao emissor continuar a enviar pacotes de dados ou de controlo, caso contrário retorna Rej e o programa vai tentar retransmitir aquela trama.

Por fim, o **llclose()** serve para terminar a ligação entre o emissor e o recetor, funciona quase igual ao **llopen()** só que o emissor envia um frame DISC, e o recetor vai receber este frame e enviar outro frame DISC, e por fim o recetor depois de receber o frame DISC, envia um frame UA e acaba com a ligação se não existir nenhum erro pelo caminho.

Protocolo de Aplicação

A camada de aplicação, basicamente interage diretamente com toda a transferência do ficheiro. Esta camada faz uso das funções da linklayer para realizar a transferência.

Após a função ***applicationlayer*** invocar a função `llopen()` e esta ser realizada sem nenhum erro. Se a role for transmissor, depois de abrir o ficheiro, vai ser enviado o pacote de controlo inicial, seguido pela escrita e envio de pacotes de dados até não ter mais para enviar, e por fim o envio do pacote de controlo final. Se a role for recetor, a função faz basicamente o contrário, lê os pacotes de controlo e também os pacotes de dados até não ter mais para ler.

Todo este funcionamento é assegurado pelas funções da applicationLayer e por 3 funções auxiliares essenciais que decidimos criar, ***sendctrlpkt()***, ***readctrlpkt()*** , ***senddtpkt()***.

A função `sendctrlpkt` trata de escrever os pacotes de controlo necessários para o início e o fim da transferência de dados, através da função `llwrite`, assegurando assim que o transmissor delimita onde começa e onde acaba a escrita de pacotes de dados.

A função `readctrlpkt` por sua vez assegura que o recetor lê os pacotes de controlo, e assim controlo quando começa e quando acaba a leitura de dados.

A função `senddtpkt` é essencial para o transmissor, que através do `llwrite` envia todos os pacotes de dados do ficheiro, e assim assegura que o ficheiro que é recebido pelo recetor é o mesmo que foi inicialmente transferido pelo transmissor se não existirem erros pelo caminho.

A leitura de pacotes de dados por parte do recetor é totalmente assegurada pela função `llread` da linkLayer que é chamada na applicationLayer num while loop que permite ler todos os pacotes enviados pelo transmissor.

Por fim, a função `applicationlayer`, invoca ainda a função da linklayer `llclose` para terminar a ligação entre o recetor e o transmissor, após estar concluída toda a escrita de pacotes de dados e de controlo, ou então por exceder o número de tentativas fracassadas.

Validação

Ao longo do projeto fomos testando alguns parâmetros para garantir que produzíamos um bom produto final.

- Transferência de diferentes ficheiros(nomes e tamanhos diferentes) para testar o `llwrite` e o `llread`.
- Transferência de ficheiros com diferentes baudrates.
- Interrupção da Porta série.
- Introdução de ruído na Porta série.
- Transferência com Baudrates e delay de propagação diferentes.

Em alguns aspetos destes testes o nosso programa falhou, como é o caso da interrupção da porta série, que algumas vezes o programa excede o número de retransmissões mesmo sem as realizar totalmente.

Eficiência do protocolo de ligação de dados

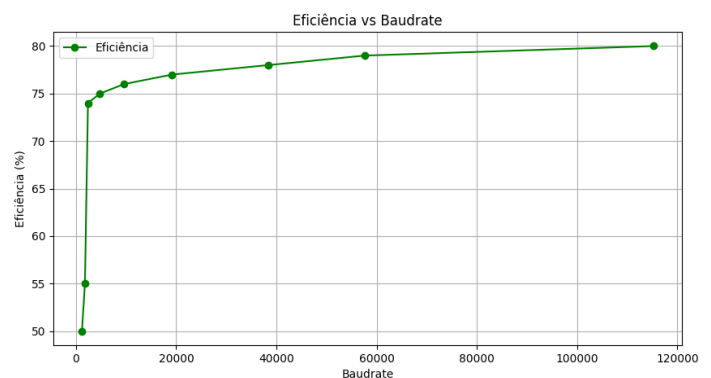
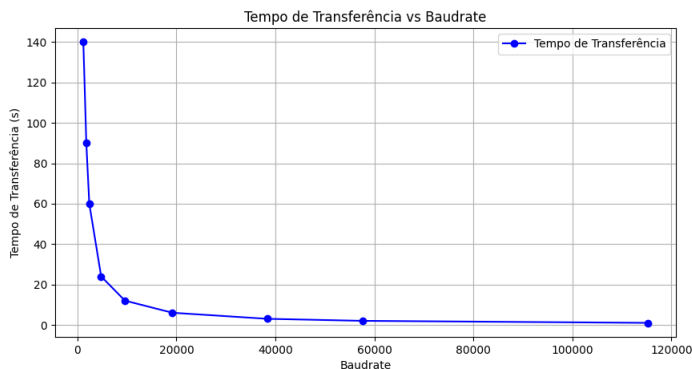
Diferentes Baudrates

O nosso pinguim tinha um tamanho de 10968 bytes e estávamos a transferir cada pacote com 1000 bytes, ao variar o baudrate da porta série conseguimos obter os seguintes dados:

baudrates = [1200, 1800, 2400, 4800, 9600, 19200, 38400, 57600, 115200]

tempos_transferencia = [140, 90, 60, 24, 12, 6, 3, 2, 1]

Calculámos ainda a eficiência com a fórmula (**Tempo útil/Tempo transferência**) -> **tempo útil = tamanho dos dados/baudrate**.



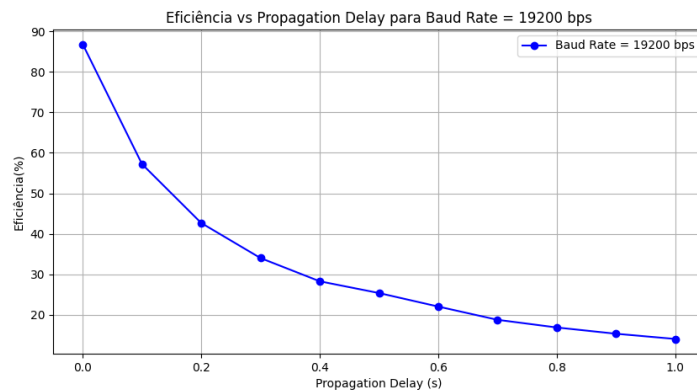
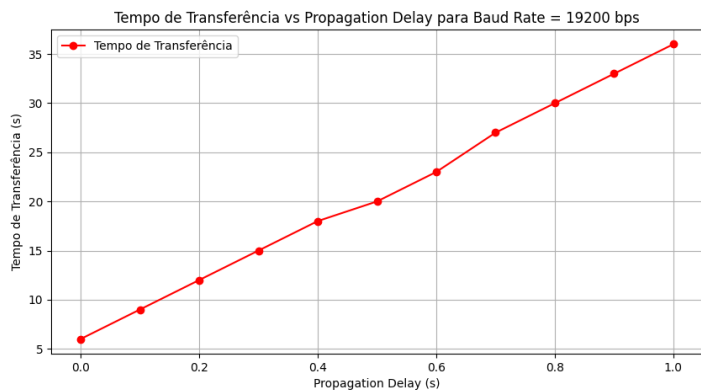
Conclusões: Com os dados obtidos, conseguimos perceber claramente que o tempo de transmissão diminui com o aumento do baudrate, ou seja, são inversamente proporcionais. Já para a eficiência, vê-se que quando o baudrate é mínimo a eficiência é muito pequena, mas com o aumento do baudrate e a partir mais ou menos dos 9600, a eficiência é alta, torna-se estável e aumenta pouco. Por fim, um aumento da baudrate proporciona um menor tempo de transferência e uma maior eficiência no nosso programa.

Diferentes PropDelays

Com um baudrate de 19200, e tramas de tamanho fixo 1000 bytes, com diferentes delays, conseguimos os seguintes dados:

propagation_delays_usec = [0, 100000, 200000, 300000, 400000, 500000, 600000, 700000, 800000, 900000, 1000000]

tempos_transferencia = [6, 9, 12, 15, 18, 20, 23, 27, 30, 33, 36]



Conclusões: Com os dados obtidos, conseguimos perceber claramente que o tempo de transmissão aumenta com o aumento do delay, ou seja, são diretamente proporcionais. Já para a eficiência, vê-se que quando o delay é mínimo a eficiência é muito grande, mas com o aumento do delay, a eficiência é muito baixa. Por fim, um aumento do delay proporciona um maior tempo de transferência e uma menor eficiência no nosso programa, ou seja, quanto maior for o delay inserido na porta série mais tempo vai demorar a transferência de dados entre o transmissor e o receptor.

Conclusões Extra:

Outros testes onde poderíamos retirar dados diferentes, seria por exemplo, testar o programa com diferentes tamanhos de tramas, ou então diferentes FER. Com o aumento dos tamanhos veríamos que o tempo de transferência iria diminuir, e a eficiência aumentar, já com o aumento da taxa de erros iríamos tirar as conclusões contrárias, ou seja, o tempo aumentava e a eficiência diminuía.

Conclusões Finais do projeto

Com a realização deste projeto, e consequente implementação e desenvolvimento de duas das partes mais importantes no protocolo de transferência de um ficheiro, **applicationLayer e linkLayer**, permitiu-nos perceber melhor conceitos ensinados e explicados com detalhe nas aulas teóricas, como o funcionamento do **Stop&Wait** e também de operações em bytes, como stuffing e destuffing.

Em geral pensamos que o projeto foi bem realizado, apesar de alguns problemas finais em certos aspetos, o produto final está sólido e com todos os protocolos e algoritmos implementados.